Computational Physics

Topic 02 : Computational Problems involving Marko Chains

Lecture 01 : Review of Markov Chains

Dr Kieran Murphy

Computing and Mathematics, SETU (Waterford).
(kieran.murphy@setu.ie)

Autumn Semester, 2026/27

Outline

- Some simple models
- Terminology and definitions

Example (Land of Oz)

Description

The Land of Oz is blessed by many things, but not by good weather. They never have two nice days in a row. If they have a nice day, they are just as likely to have snow as rain the next day. If they have snow or rain, they have an even chance of having the same the next day. If there is change from snow or rain, only half of the time is this a change to a nice day.

Example (Land of Oz)

Description

The Land of Oz is blessed by many things, but not by good weather. They never have two nice days in a row. If they have a nice day, they are just as likely to have snow as rain the next day. If they have snow or rain, they have an even chance of having the same the next day. If there is change from snow or rain, only half of the time is this a change to a nice day.

Formalisation . . . as a transition matrix

$$\begin{array}{cc} & \\ & \text{. . . to . . .} \\ \text{. . . from . . .} & \end{array} \begin{array}{c} \begin{array}{ccc} \text{Rain} & \text{Nice} & \text{Snow} \end{array} \\ \begin{pmatrix} 1/2 & 1/4 & 1/4 \\ 1/2 & 0 & 1/2 \\ 1/4 & 1/4 & 1/2 \end{pmatrix} \end{array} \begin{array}{l} \text{Rain} \\ \text{Nice} \\ \text{Snow} \end{array}$$

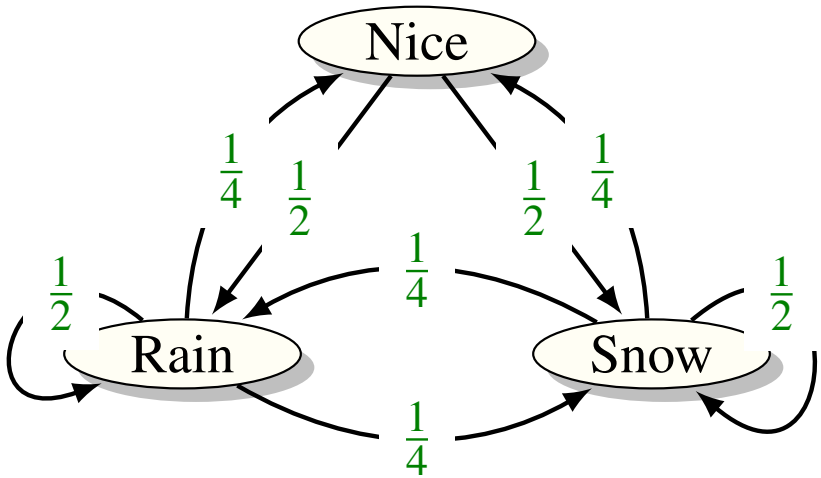
Example (Land of Oz)

Description

The Land of Oz is blessed by many things, but not by good weather. They never have two nice days in a row. If they have a nice day, they are just as likely to have snow as rain the next day. If they have snow or rain, they have an even chance of having the same the next day. If there is change from snow or rain, only half of the time is this a change to a nice day.

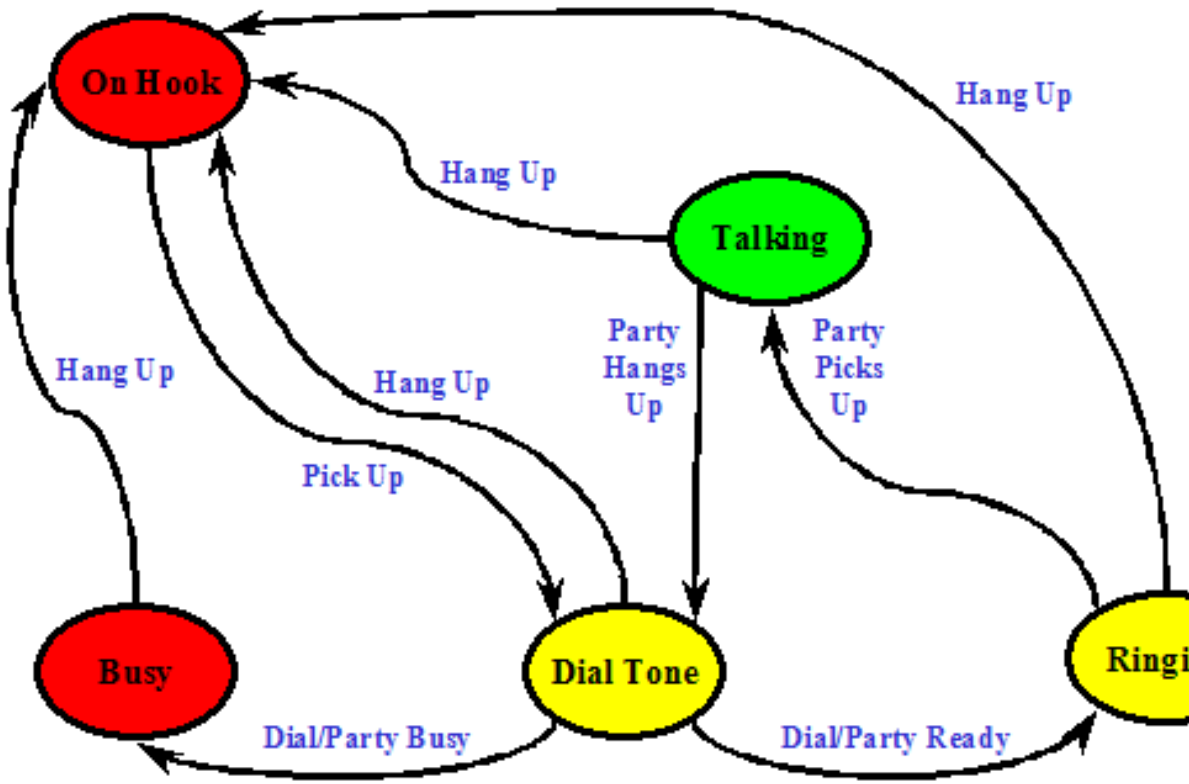
Formalisation . . . as a transition matrix . . . and finite state machine

$$\begin{array}{cc} & \begin{array}{ccc} \multicolumn{3}{c}{\ldots \text{ to } \ldots} \\ \text{Rain} & \text{Nice} & \text{Snow} \end{array} \\ \begin{array}{c} \ldots \text{from} \ldots \\ \text{Rain} \\ \text{Nice} \\ \text{Snow} \end{array} & \begin{pmatrix} 1/2 & 1/4 & 1/4 \\ 1/2 & 0 & 1/2 \\ 1/4 & 1/4 & 1/2 \end{pmatrix} \end{array}$$



Example (Model Based Testing)

Model-Based Testing*is the automatic generation of efficient test procedure sequences — using finite state machines or Markov chains – to represent a system's requirements and specified functionality.



*Google "Model Based Testing" or read Harry Robinson's (Microsoft) paper http://www.ecs.csun.edu/ rlingard/COMP595VAV/GraphTheory Techniques In Model-Based Testing.pdf

Example (Worker Employment Model) I

Example 1

Consider a worker who, at any given month, t, is either unemployed (state 0) or employed (state 1). Suppose that, over a one month period:

- An unemployed worker finds a job with probability $\alpha \in (0, 1)$.
- An employed worker loses their job and becomes unemployed with probability $\beta \in (0, 1)$.

Formulate model as a Markov chain[†]. Then:

1. What is the average duration of unemployment?
2. Over the long-run, what fraction of time does a worker find themself unemployed?
3. Conditional on employment, what is the probability of becoming unemployed at least once over the next 12 months?

[†]Since this model has only two states we could model this using the geometric distribution.

Python Implementation I

Python setup

1
```
import numpy as np
import matplotlib.pyplot as plt
from numpy.random import default_rng
rng = default_rng(12)
```

Define states

2
```
# label states for output
labels = np.array(["Rain","Nice","Show"
states = np.arange(0, len(labels))
print("Labels:", labels)
print("States:", states)
```

```
Labels: ['Rain' 'Nice' 'Show']
States: [0 1 2]
```

Python Implementation II

Define transition matrix, P

3
```
# transition probability matrix
P = np.array([[1/2, 1/4, 1/4], [1/2, 0, 1/2], [1/4, 1/4, 1/2] ])
print("Transition probabilities:\n", P)
```

Compute distributions

4
```
u0 = np.array([1,0,0])
print("Stage 0:", u0)
print("Stage 1:", np.dot(u0,P))

u0 = np.array([0,1,0])
u2 = np.dot( np.dot(u0,P), P)
print("\nStage 2:", u2)
u2 = np.dot(u0, np.linalg.matrix_power(P,2))
print("or directly by P squared:", u2)
```

Python Implementation II

Define transition matrix, P

5
```
# transition probability matrix
P = np.array([[1/2, 1/4, 1/4], [1/2, 0, 1/2], [1/4, 1/4, 1/2] ])
print("Transition probabilities:\n", P)
```

Compute distributions

6
```
# transition probability matrix
P = np.array([[1/2, 1/4, 1/4], [1/2, 0, 1/2], [1/4, 1/4, 1/2] ])
print("Transition probabilities:\n", P)
```

Python Implementation III